Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Methods for Detecting Attack

Final Project

Submitted by

Natalie Mirelashvili
Faculty of Computer and Information Science
Ben-Gurion University of the Negev
E-mail: miralshv@post.bgu.ac.il

[complete others]

June 13, 2026

Abstract

Large language model (LLM) agents are increasingly augmented with tools, persistent memory, retrieval components, and the ability to communicate with other agents, which extends their capabilities but also broadens their attack surface. This is particularly true for multi-agent systems, where sensitive information can propagate through internal channels – messages between agents, tool arguments, memory writes, and logs – before, or even without, reaching the final user-facing output. This project proposes an AutoResearch-driven red-teaming methodology for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes attack variants, evaluates them against a target system on a fixed task, scores the outcome, and retains improving variants. We instantiate two independent optimization loops, one maximizing synthetic sensitive-information exposure and the other maximizing task-utility degradation and operational-cost amplification, and evaluate them across AgentDojo and three multi-agent frameworks: AutoGen, CrewAI, and LangGraph. By measuring both final-output and internal-channel leakage, utility degradation, cost amplification, and cross-framework transferability, we aim to determine whether automatically optimized attacks outperform random and manually designed baselines, and whether attacks discovered on one framework generalize to others.

1 Introduction

Large language model (LLM)-based agents are increasingly used for tasks that go beyond single-turn question answering: they call external tools and APIs, retrieve and incorporate documents, read and write persistent memory, and plan a sequence of actions toward a goal. In multi-agent systems, several such agents are composed into a single pipeline, for example, a planner that decomposes the task, a retriever or tool-using agent that gathers external information, a worker agent that carries out the core task, and a reviewer or guard agent that checks the result before it is returned to the user.
Although this decomposition improves modularity and often enhances task performance, it also introduces internal communication channels, such as inter-agent messages, tool-call arguments, memory writes, and logs, that are not directly visible to the end user.

Existing evaluations of prompt-injection robustness for LLM-integrated systems typically focus on whether the final, user-facing response leaks sensitive content or deviates from the intended task [1, 2]. In an agentic or multi-agent pipeline, however, a sensitive value can pass through, and be exposed at several internal points before any final answer is produced. An attacker who injects malicious content into a retrieved document, a tool's output, a task input, or a message exchanged between agents may cause a secret to surface in a tool argument, an inter-agent message, or a memory entry, even when the system's final response to the user appears unremarkable. Evaluating only the final

output can therefore substantially underestimate how exposed such a system really is, and may also miss attacks that degrade task performance or increase operational cost without leaking anything at all.

This project investigates that gap directly. We ask (i) whether an automated, iterative red-teaming process can discover attacks that exploit the internal channels of agentic and multi-agent systems more effectively than random or manually designed attacks, and (ii) whether final-output-only evaluation indeed misses leakage and degradation that an internal-channel-aware evaluation would catch. Rather than hand-crafting every attack variant, we adopt an AutoResearch-style loop, described below: an LLM agent proposes an attack variant, the variant is run against a target agentic system on a fixed task, the outcome is scored against a chosen metric, and the variant is kept or discarded before the next is proposed. We instantiate this loop twice, for two complementary attacker goals: maximizing the rate at which a synthetic canary secret is exposed through any channel, final output or internal, and degrading task utility and/or amplifying operational cost (extra tool calls, latency, tokens) without necessarily leaking a secret. We study both loops across four systems – AgentDojo, AutoGen, CrewAI, and LangGraph, configured with a shared abstract pipeline so that results are comparable across frameworks. Throughout, the attacker only injects content into untrusted channels (retrieved documents, tool outputs, task inputs, inter-agent messages), no real user data is involved and all sensitive information is represented by synthetic canary secrets in sandboxed tasks.

1.1 Agentic and Multi-Agent LLM Systems

An agentic LLM system augments a base language model with the ability to call external tools or APIs, retrieve and incorporate external documents, read and write to a memory store, and plan over multiple steps rather than respond in a single turn. AgentDojo [3] is one such environment: it provides realistic tool-using tasks together with an established benchmark for prompt-injection attacks and defenses, and serves in this project as a single-agent, tool-using reference point.

Multi-agent frameworks compose several agents of this kind into a single pipeline. AutoGen [4] structures this composition as a conversation in which agents exchange messages and may invoke tools or request human input. CrewAI [5] assigns each agent an explicit role, goal, and set of tools, and lets agents delegate subtasks to one another. LangGraph [6] represents the pipeline as an explicit graph of nodes and edges with shared, inspectable state, which makes it convenient to trace exactly where in the pipeline an attack takes effect. In all three frameworks, the additional roles: planner, retriever/tool agent, worker, reviewer/guard communicate through messages, shared memory, or delegation calls that never appear in the final response shown to the user, but that attacker-controlled input may still be able to influence or read from. The threat model introduced later in this

paper targets exactly these channels.

1.2 AutoResearch-Style Optimization

AutoResearch [7] denotes an iterative research-automation pattern in which an LLM agent edits an experimental configuration, prompt, or piece of code, runs an evaluation, scores the outcome against a target metric, and keeps the change only if it improves that metric, otherwise discarding it and proposing a different change. The pattern was originally proposed to automate iterative improvement of training and evaluation pipelines, but the underlying generate-evaluate-score-keep/discard cycle does not depend on what is being optimized.

This project adapts that cycle to automated red teaming. Here, the LLM agent searches not for configurations that improve benign task performance, but for attack variants changes to the injected content, its placement within the pipeline, the targeted agent, and its phrasing. That increase a chosen attack-effectiveness metric against a fixed target system and task. This differs from recent reinforcement-learning-based approaches to automated prompt injection, which optimize universal adversarial suffixes through gradient-based or policy-gradient training over model weights [8, 9]. Because the AutoResearch loop operates on natural-language attack variants that are proposed and judged by an LLM agent rather than on model parameters, it requires no gradient access and can, in principle, be applied directly to closed, black-box agentic systems.

1.3 Contributions

This project makes the following contributions:

- We instantiate an AutoResearch-style red-teaming loop for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes, evaluates, and selects attack variants against a fixed target system and task.
- We define two independent optimization objectives for this loop: maximizing the total exposure rate of a synthetic canary secret across final-output and internal channels, and maximizing a weighted combination of task-utility drop and operational-cost amplification.
- We define an evaluation methodology that separately measures final-output leakage and internal-channel leakage (in tool arguments, inter-agent messages, memory writes, and logs), enabling a direct test of whether output-only evaluation underestimates real exposure.
- We apply this methodology to four systems spanning single-agent (AgentDojo) and multi-agent (AutoGen, CrewAI, LangGraph) architectures configured with a shared

abstract workflow, and study whether attacks optimized on one framework transfer to the others.

- We compare AutoResearch-optimized attacks against clean, random, and manually designed baselines, under a range of prompt-based, guard-agent, redaction, and tool-output-filtering defenses.

2 Related Work

2.1 Prompt injection and automated attack generation

Prompt injection research studies the failure mode in which LLM-integrated applications process trusted instructions and untrusted content within the same model context. Early work on indirect prompt injection demonstrated that malicious instructions embedded in external sources, such as web pages or documents, can later be activated when an application retrieves or summarizes that content [1]. Subsequent formalization and benchmarking work modeled prompt injection as a security problem for LLM-integrated applications and evaluated both attacks and defenses across different task settings [2]. Robustness studies further showed that model behavior can be sensitive to small adversarial perturbations at the character, word, sentence, or semantic level, indicating that injection success depends not only on the malicious goal but also on prompt structure, task framing, and the placement of attacker-controlled text [10].

As LLM applications became more tool-using and agentic, evaluation moved from single prompts toward benchmarked attack environments. AgentDojo [3] provides a dynamic environment for evaluating prompt-injection attacks and defenses in tool-using agents, with tasks that require agents to follow user goals while resisting conflicting instructions from untrusted data sources. This line of work is important because attacks against agentic systems are often measured by effects on actions, tool calls, or task completion rather than by textual compliance alone. More generally, automated attack frameworks reduce reliance on hand-crafted prompts by repeatedly generating candidates, executing them against a target behavior, and scoring the observed outcome [2, 3].

Optimization-based attacks further illustrate how adversarial prompts can be found automatically rather than manually authored. Universal and transferable adversarial attacks on aligned language models showed that search over suffix tokens can produce attacks that generalize across multiple models, including black-box targets [11]. JudgeDeceiver [12] extended optimization-based prompt injection to LLM-as-a-Judge systems, where the attack objective is to manipulate an evaluator into preferring a target response. These studies established automated prompt search as a practical adversarial method and showed that successful attacks may be short, reusable, and difficult to detect

from surface form alone [11, 12].

Recent work has extended automated attack generation with reinforcement learning, making the attack process especially relevant for multi-step and agentic settings. AutoInject [8] formulates prompt injection as an RL problem and learns universal, transferable suffixes while jointly optimizing attack success and benign-task utility. This framing allows attack strategies to be learned from reward signals rather than designed entirely by human prompt engineers. Slingshot [9] applies a related RL approach to agent-to-agent jailbreaking, using verifiable tool execution as the success signal instead of relying only on subjective judgments of textual compliance. Together, these works position reinforcement learning as a mechanism for discovering adversarial behavior when the target system exposes structured feedback through task outcomes, tool use, or other observable effects [8, 9].

2.2 Defenses for LLM-integrated systems

Defenses against prompt injection are commonly grouped into soft defenses and hard defenses [2, 13]. Soft defenses operate at inference time or around the prompt without changing the underlying model or application architecture, while hard defenses modify the system design, data flow, or model training procedure to impose stronger trust boundaries [2, 14]. This distinction is useful because it separates lightweight mitigations that can be deployed quickly from structural approaches that require deeper integration.

Soft defenses

Soft defenses include prompt rewriting, instruction reminders, input/output filtering, detection heuristics, and other inference-time safeguards [2, 15]. In formal benchmarking of prompt injection defenses, prompt-level mitigations and detector-style defenses were shown to be practical baselines, but their effectiveness varied substantially across attack settings and model behavior [2]. For indirect prompt injection, BIPIA introduced boundary-awareness mechanisms and explicit reminders that encourage models to treat retrieved content as data rather than as executable instructions [15]. These methods are attractive because they are inexpensive to add to existing applications, yet benchmark results show that prompt-only protections can be fragile when attacks are adaptive or when the model does not consistently preserve the intended instruction hierarchy [2, 15].

Hard defenses

Hard defenses address prompt injection by changing how an LLM-integrated application represents authority, separates context, or validates information sources [14, 16]. StruQ [14] treats prompt injection as a structured-query problem: instructions and data are placed in separate channels, and the model is trained to follow only the instruction

channel. Other system-level mitigation work emphasizes identifying vulnerabilities in the application design itself, including cases where untrusted content can influence privileged instructions or tool actions [16]. Survey work situates these approaches within a broader defense toolbox that includes corpus cleaning, robustness optimization, safe instruction tuning, and privacy-oriented controls [13]. Overall, the literature shows a gradual shift from isolated prompt-level safeguards toward defenses that make trust boundaries, data provenance, and tool-use authority explicit parts of the application architecture [14, 16, 13].

3 Motivation and Goals

Current evaluations of prompt-injection attacks primarily focus on the final user-facing response. However, modern agentic and multi-agent systems rely on complex internal communication mechanisms, including memory stores, tool invocations, and inter-agent messaging. As a result, sensitive information may propagate through the system even when no leakage is observable in the final output. This raises concerns that existing evaluation methodologies may underestimate both attack impact and defense effectiveness. Furthermore, manually designing attacks for such systems is labor-intensive and may fail to uncover subtle attack strategies. These challenges motivate the need for automated attack discovery and evaluation methodologies that explicitly account for internal system behavior.

Research Questions:

RQ1: Can AutoResearch-optimized attacks outperform random and manually designed baselines in exposing synthetic secrets, degrading task utility, or amplifying operational cost?

RQ2 To what extent does final-output-only evaluation underestimate leakage and attack impact compared with evaluation that also monitors internal channels?

RQ3 Do attacks optimized on one agentic or multi-agent framework transfer to other frameworks?

4 Threat Model

4.1 Attacker Capabilities

We assume a controlled research setting in which the attacker can inject malicious or misleading content into one or more untrusted channels, such as:

- Retrieved documents.

- Tool outputs.
- Emails or task inputs.
- Inter-agent messages.
- External knowledge sources.

The attacker does not access real private data. All sensitive information used in the experiment is synthetic and represented using canary secrets.

4.2 Attack Goals

We consider two attack goals.

4.2.1 Sensitive Information Leakage

The attacker attempts to cause the system to expose a synthetic secret, either in the final output or in an internal system channel.

4.2.2 Performance Degradation

The attacker attempts to reduce task performance or increase operational cost without necessarily leaking a secret.

4.3 Out of Scope

This work does not target real users, real private data, production systems, credential theft, or unauthorized access. All experiments are performed in sandboxed environments using synthetic secrets and controlled tasks.

5 Experimental Systems

We evaluate the framework on four systems.

5.1 AgentDojo

AgentDojo serves as the benchmark-oriented baseline environment. It provides realistic tool-using agent tasks and is suitable for evaluating prompt-injection robustness, utility preservation, and attack success.

5.2 AutoGen

AutoGen represents a conversational multi-agent framework in which agents interact through messages and may use tools or human-in-the-loop workflows.

5.3 CrewAI

CrewAI represents role-based multi-agent collaboration, where agents are assigned goals, roles, and tools, and may delegate subtasks to one another.

5.4 LangGraph

LangGraph represents stateful graph-based agent workflows. It is especially useful for tracing where an attack succeeds, because intermediate nodes, edges, state updates, and tool calls can be logged explicitly.

5.5 Unified Experimental Scenario

To enable comparison, each framework is configured to implement a similar abstract workflow:

User Task → Planner Agent → Retriever/Tool Agent → Worker Agent → Reviewer/Guard Agent → Final Output

Figure 1: Unified multi-agent workflow used across frameworks.

6 AutoResearch Red-Teaming Framework

6.1 Overview

The proposed framework uses AutoResearch-style optimization to generate, test, and select attack variants. Each loop receives a target system, a task, an attack space, and a scoring function.

Generate Attack Variant → Run Evaluation → Score Result → Keep or Reject → Next Iteration

Figure 2: AutoResearch red-teaming loop.

6.2 Loop A: Leakage Optimization

The first loop optimizes for sensitive-information leakage.

$$\text{Objective}_{leak} = \max\left(\text{TotalExposureRate}\right) \tag{1}$$

The loop may modify:

- Attack prompt wording.
- Injection location.
- Target agent.
- Tool-output content.
- Retrieved document content.
- Canary placement.
- Attack stealth level.

6.3 Loop B: Performance Degradation Optimization

The second loop optimizes for reduced utility or increased cost.

$$\text{Objective}_{perf} = \max\left(\alpha \cdot \text{UtilityDrop} + \beta \cdot \text{CostAmplification}\right) \tag{2}$$

where α and β control the relative importance of correctness degradation and cost increase.

The loop may modify:

- Misleading documents.
- Ambiguous task inputs.
- Tool outputs.
- Delegation-inducing instructions.
- Retry-inducing perturbations.
- Conflicting context.

7 Evaluation Metrics

7.1 Leakage Metrics

7.1.1 Leakage Attack Success Rate

$$\text{Leakage ASR} = \frac{\#\text{runs with leakage}}{\#\text{attacked runs}} \tag{3}$$

7.1.2 Final-Output Leakage Rate

$$\text{FinalOutputLeakage} = \frac{\#\text{runs with leakage in final output}}{\#\text{attacked runs}} \tag{4}$$

7.1.3 Internal Leakage Rate

$$\text{InternalLeakage} = \frac{\#\text{runs with leakage in internal channels}}{\#\text{attacked runs}} \tag{5}$$

7.1.4 Total Exposure Rate

$$\text{TotalExposureRate} = \frac{\#\text{runs with leakage in any channel}}{\#\text{attacked runs}} \tag{6}$$

7.1.5 Output-Only Miss Rate

$$\text{OutputOnlyMissRate} = \frac{\#\text{runs with internal leakage but no final-output leakage}}{\#\text{attacked runs}} \tag{7}$$

7.2 Performance Metrics

7.2.1 Utility Drop

$$\text{UtilityDrop} = \text{CleanTaskSuccessRate} - \text{AttackedTaskSuccessRate} \tag{8}$$

7.2.2 Cost Amplification

$$\text{CostAmplification} = \frac{\text{AttackedCost}}{\text{CleanCost}} \tag{9}$$

7.2.3 Latency Increase

$$\text{LatencyIncrease} = \frac{\text{AttackedLatency}}{\text{CleanLatency}} \tag{10}$$

7.2.4 Tool Call Increase

$$\text{ToolCallIncrease} = \frac{\text{AttackedToolCalls}}{\text{CleanToolCalls}} \tag{11}$$

7.3 Transferability Metrics

7.3.1 Transfer ASR

$$\text{TransferASR} = \frac{\text{ASR on unseen systems}}{\text{ASR on source system}} \tag{12}$$

7.3.2 Generalization Gap

$$\text{GeneralizationGap} = \text{ASR}_{seen} - \text{ASR}_{unseen} \tag{13}$$

7.4 Defense Metrics

- Leakage reduction.
- Utility preservation.
- False refusal rate.
- Over-defense rate.
- Defense cost overhead.
- Attack success despite guard.

8 Experimental Design

8.1 Baselines

We compare four attack settings:

- **B0: Clean**: no attack.
- **B1: Random**: random or naive injection.
- **B2: Manual**: manually designed attack.
- **B3: AutoResearch**: automatically optimized attack.

8.2 Defense Settings

We evaluate the following defense configurations:

- **D0: No defense**.
- **D1: Prompt-based defense**.

Table 1: Experimental matrix.

System	Architecture	Goal	Attack Type	Defense
AgentDojo	Single-agent/tool-using	Leakage	Clean / Random / Manual / AutoResearch	D0–D4
AutoGen	Multi-agent	Leakage	Clean / Random / Manual / AutoResearch	D0–D4
CrewAI	Multi-agent	Degradation	Clean / Random / Manual / AutoResearch	D0–D4
LangGraph	Multi-agent graph	Degradation	Clean / Random / Manual / AutoResearch	D0–D4

Table 2: Sensitive-information leakage results.

System	Attack	Leakage ASR	Final Output	Internal Leakage	Total Exposure	Ou
AgentDojo	Random	TODO	TODO	TODO	TODO	
AgentDojo	Manual	TODO	TODO	TODO	TODO	
AgentDojo	AutoResearch	TODO	TODO	TODO	TODO	
AutoGen	AutoResearch	TODO	TODO	TODO	TODO	
CrewAI	AutoResearch	TODO	TODO	TODO	TODO	
LangGraph	AutoResearch	TODO	TODO	TODO	TODO	

- **D2: Guard agent**.
- **D3: Memory or inter-agent redaction**.
- **D4: Tool-output filtering**.

8.3 Experimental Matrix

Each experiment is defined by:

$$E = (S, G, A, D, T) \tag{14}$$

where S is the system, G is the attack goal, A is the attack type, D is the defense setting, and T is the task/domain.

9 Results

This section will present the empirical results.

Table 3: Performance degradation results.

System	Attack	Task Success	Utility Drop	Cost Amp.	Latency Inc.	Tool Call Inc
AgentDojo	Random	TODO	TODO	TODO	TODO	TODO
AgentDojo	Manual	TODO	TODO	TODO	TODO	TODO
AgentDojo	AutoResearch	TODO	TODO	TODO	TODO	TODO
AutoGen	AutoResearch	TODO	TODO	TODO	TODO	TODO
CrewAI	AutoResearch	TODO	TODO	TODO	TODO	TODO
LangGraph	AutoResearch	TODO	TODO	TODO	TODO	TODO

Table 4: Cross-system transferability results.

Source System	Target System	Goal	Transfer ASR	Generalization Gap
AgentDojo	LangGraph	Leakage	TODO	TODO
AgentDojo	CrewAI	Leakage	TODO	TODO
LangGraph	AutoGen	Leakage	TODO	TODO
AgentDojo	LangGraph	Degradation	TODO	TODO
LangGraph	CrewAI	Degradation	TODO	TODO

9.1 Leakage Results

9.2 Performance Degradation Results

9.3 Transferability Results

9.4 Defense Results

10 Discussion

10.1 Final Output vs. Internal Exposure

This section discusses whether final-output-only evaluation misses important leakage channels. We expect this gap to be especially relevant in multi-agent systems, where internal messages and tool arguments may expose sensitive information even when the final answer is safe.

10.2 AutoResearch vs. Manual Attacks

This section compares whether AutoResearch-optimized attacks outperform manual and random baselines.

Table 5: Defense evaluation.

System	Defense	Leakage Reduction	Utility Preservation	False Refusal	Cost Overh
AgentDojo	Prompt defense	TODO	TODO	TODO	TO
AgentDojo	Guard agent	TODO	TODO	TODO	TO
LangGraph	Redaction	TODO	TODO	TODO	TO
CrewAI	Tool filtering	TODO	TODO	TODO	TO

10.3 Transferability Across Frameworks

This section analyzes whether attack variants optimized on one framework generalize to other frameworks, domains, or defense settings.

10.4 Security-Utility Tradeoff

This section analyzes whether defenses reduce leakage without significantly degrading task utility or increasing operational cost.

11 Limitations

This work has several limitations:

- The experiments use synthetic secrets rather than real private data.
- Framework implementations may not reflect all production deployments.
- AutoResearch search quality depends on the defined attack space and scoring function.
- Some semantic leakage may be difficult to detect using exact canary matching alone.
- Results may vary across LLM providers and model versions.

12 Ethical Considerations

All experiments are conducted in controlled environments using synthetic data. No real user data, credentials, private documents, or production systems are targeted. The goal of this work is to improve the evaluation and defense of agentic and multi-agent systems.

13 Conclusion

This paper proposes an AutoResearch-driven methodology for automated red teaming of agentic and multi-agent systems. The framework evaluates both sensitive-information

leakage and performance degradation, while measuring transferability across AgentDojo, AutoGen, CrewAI, and LangGraph. By inspecting both final outputs and internal channels, the proposed evaluation aims to reveal exposure that output-only testing may miss. The expected outcome is a systematic framework for comparing vulnerabilities, defenses, and generalization behavior across modern agent architectures.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pp. 79–90, 2023.

[2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, (Philadelphia, PA), pp. 1831–1847, USENIX Association, Aug. 2024.

[3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, 2024.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," 2023.

[5] CrewAIInc, "Crewai: Framework for orchestrating role-playing, autonomous ai agents." `https://github.com/crewAIInc/crewAI`, Nov. 2023.

[6] LangChain AI, "Langgraph: Building stateful, multi-actor applications with llms." `https://github.com/langchain-ai/langgraph`, Jan. 2024.

[7] A. Karpathy, "Autoresearch: A bounded, public loop for automated machine learning experimentation." `https://github.com/karpathy/autoresearch`, 2026.

[8] X. Chen, J. Zhang, and F. Tramer, "Learning to inject: Automated prompt injection via reinforcement learning," *arXiv preprint arXiv:2602.05746*, 2026.

[9] S. Nellessen and T. Kachman, "David vs. goliath: Verifiable agent-to-agent jailbreaking via reinforcement learning," *arXiv preprint arXiv:2602.02395*, 2026.

[10] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, "Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts," 2023.

[11] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023.

[12] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to llm-as-a-judge," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.

[13] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.

[14] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," 2024. To appear at USENIX Security Symposium 2025.

[15] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '25, Association for Computing Machinery, 2025.

[16] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, "Identifying and mitigating vulnerabilities in llm-integrated applications," 2023.